

iwmQT Trading Terminal

Four days of build progress on the institutional trading terminal — what's live, what's deliberately simulated, why the data layer is split the way it is, and what changes once this points at a real market feed and a real risk book.

Akash Rai akash.rai@iwmquant.com 24 Aug 2026 branch `feature/dev`

5

BUILD COMMITS

5

ROLES WIRED

4

POSTGRES TABLES

1

LIVE WS CHANNEL

The scaffolding is in — end to end.

In four days I built the full skeleton of an institutional trading terminal: authentication and role-based access, a Postgres-backed order and position schema, a first-pass pre-trade risk engine, five role-specific dashboards, and a live WebSocket channel streaming market ticks. Every layer a real trading day touches now exists and is wired together in the UI — not just designed on paper.

What's still ahead is deliberate, not accidental: the market feed, order execution, and most RMS rules are stubbed until this points at real infrastructure. Slides 3–7 walk through what's built and why it was built in that order; slide 8 is the honest inventory of what's real versus simulated; slides 9–12 cover the database reasoning and what changes for production.

Foundation: auth & access control

WHAT

Signup restricted to `@iwmquant.com`, gated by an email OTP. Passwords hashed with `bcrypt (cost 12)`. Sessions are a JWT in an `httpOnly` cookie. Accounts lock after `5 failed logins`; passwords expire after 15 days. Five roles — `SUPER_ADMIN`, `RMS_ADMIN`, `PM`, `COMPANY_ACCOUNT`, `TRADER` — enforced on every route.

HOW

The JWT is signed server-side and set via `Set-Cookie`, never handed to client JS — an XSS can't read it. An `authorize(...roles)` middleware checks role before any controller runs, and the client mirrors the same check per route so the UI never even offers a control the API would reject.

WHY

Feature work can't outrun access control on a trading desk. Order entry, RMS admin, and super-admin screens all depend on knowing exactly who's asking — building this first means every later feature inherits the guardrail instead of having one bolted on afterward.

Core OMS skeleton: the Postgres schema

WHAT

Four tables — `users`, `orders`, `banned_scripts`, `otp_verifications` — with CHECK constraints that make an invalid order unrepresentable: a LIMIT order without a price, or a MARKET order with one, is rejected by the database itself.

HOW

Orders are written as `PENDING` and read back through indexes on `(user_id, created_at)` and `status`. Positions are computed live by aggregating executed orders rather than kept as a separately maintained table, for now.

WHY

Order state is transactional by nature — it must never be silently lost, duplicated, or half-written. Postgres's row-level constraints and ACID guarantees catch bad state at write time, before it ever reaches a trader's screen. This choice is the setup for slide 9.

RMS pre-trade ban engine, v1

WHAT

Every order placement is checked against a symbol blocklist before it's written. A banned symbol is rejected with the ban reason returned to the trader. RMS_ADMIN / SUPER_ADMIN can ban or unban a symbol from the admin dashboard.

HOW

One indexed lookup — `SELECT * FROM banned_scripts WHERE symbol = $1` — runs inline in the order-placement route, ahead of the insert.

WHY

This is the smallest possible version of pre-trade risk on purpose: one unambiguous gate, so the order-placement path and the ban/unban admin workflow both prove out before layering in exposure, margin, and circuit-breaker rules — which need real market and account data to mean anything (see slide 11).

Role-based dashboards

WHAT

Five distinct shells — Trader, PM, RMS Admin, Company, Super Admin — each gated by role. Super Admin can change any user's role and lock/unlock accounts live against Postgres.

HOW

A client-side route guard checks the session role against an allow-list per route; the server's `authorize()` double-checks the same role on every mutating endpoint, so the gate can't be bypassed by calling the API directly.

WHY

Different desk functions need different tools and different blast radius — a PM should never see the controls compliance sees. Wiring all five shells now let us validate the navigation and permission model against every role at once, instead of relitigating it role by role later.

Real-time market data plumbing

WHAT

A dedicated WebSocket server at `/ws/market-data` pushes a snapshot on connect, then a tick every 600ms. The Watchlist UI subscribes live and shows connected / disconnected state.

HOW

A plain `ws` server attached to the same HTTP server as the REST API — no socket.io, no broker in front of it yet, one process.

WHY

The transport is the deliverable here, not the data. Ticks are synthetic on purpose — the goal was to prove the push model (server-initiated updates, reconnects, live UI binding) end to end before a real feed exists to plug into it. Swapping the generator for a real feed handler shouldn't require touching the client at all (slide 10).

What's real, what's simulated, what's not built

Every module below is visible and clickable in the UI today. This is which ones are trustworthy right now.

SUBSYSTEM	STATUS	NOTE
Auth & role-based access	● LIVE	Real sessions, real DB-backed roles.
Order placement / cancel	● LIVE	Writes real rows — but no order ever leaves PENDING.
RMS symbol ban	● LIVE	Real blocklist, checked on every order.
Market data feed	● SIMULATED	Random-walk generator over 6 hardcoded instruments.
Positions	● NOT MEANINGFUL YET	Aggregates EXECUTED orders — a state nothing reaches yet.
P&L	● STUB	Hardcoded ₹0.00 placeholder in the grid.
Strategy Panel	● STUB	Local UI state only — no backend route exists.
Log Window	● STUB	Fake heartbeat timer, not sourced from real events.
Kill switch / audit log / OMS config	● STUB	Dashboard cards with no route behind them.
Company data model	● STUB	No companies table — company_id is free text today.

None of this is a gap I'd apologize for — it's the order I'd build a trading system in: prove

the rails before wiring anything that moves money.

Why Postgres, if ClickHouse is what HFT runs on?

We don't have to choose one — and the app already doesn't. Each database is doing the job it's built for.

OLTP Postgres Storage model Row-oriented, ACID transactions, row-level locking. Best at Point lookups and updates by key — read one row, change it, guarantee it stuck. Used here for <code>users</code>, <code>orders</code>, <code>banned_scripts</code> — anything that gets updated and must never be lost or duplicated.	OLAP ClickHouse Storage model Columnar, append-only, updates are async background rewrites. Best at Bulk range scans over billions of rows, ingesting at very high throughput. Used here for <code>auth_events</code> today — an append-only audit log, ordered by time, never mutated.
---	---

A single order's `PENDING → EXECUTED` transition needs a transaction and a constraint check — ClickHouse's async mutations are the wrong tool for that. Conversely, ingesting thousands of ticks per second into Postgres and then scanning them for backtesting would run an order of magnitude heavier than a column store built for exactly that scan pattern. The split we already have — Postgres for state, ClickHouse for append-only history — is the right shape; it just needs extending to trade data, not replacing.

Once there's a real feed and a real order route

01 Replace the mock generator

Swap the `setInterval` random-walk in `marketData.service.js` for an adapter onto the real broker/exchange feed.

02 Ticks land in ClickHouse, not Postgres

New schema — symbol, exchange timestamp, ingest timestamp, LTP, bid/ask, volume, OI — as a MergeTree partitioned by day, ordered by (symbol, timestamp).

03 Batch the inserts

ClickHouse wants bulk writes, not one row at a time. A queue (Kafka or a Redis stream) between the feed handler and the ClickHouse writer absorbs burst rate and lets ingestion and storage scale independently.

04 Cache the current price

RMS and the UI both need "the latest LTP" in microseconds. Scanning ClickHouse per order is too slow for a pre-trade check — that number lives in Redis.

05 Wire real execution

Integrate a broker/exchange order API so orders actually transition PENDING → EXECUTED / PARTIALLY_FILLED / REJECTED via execution reports, instead of sitting in PENDING forever.

06 Split orders from fills

Add a `fills` table — one immutable row per fill — separate from mutable order state. Real positions and P&L get computed from fills, not from raw order rows.

The ban list becomes a real risk engine

01 Move checks out of the database round-trip

An in-memory rules cache (Redis, refreshed from Postgres) replaces the per-order query — a real desk needs sub-millisecond pre-trade checks at real order rates.

02 Expand past a symbol blocklist

Margin and exposure limits per user and company, position limits per symbol, price-band / circuit checks against the live LTP cache, order-rate throttling, and a real kill switch — today these are dashboard cards with nothing behind them.

03 Audit every decision

Every RMS decision and order-lifecycle transition should land in the same append-only pattern already used for `auth_events`. Right now only login/logout is captured — nothing about trades or risk is.

What the schema and the deployment need

01 A real companies table

With a proper foreign key from `users.company_id` — it's free text today, and the COMPANY_ACCOUNT role and "entity-wide" UI copy already assume it exists.

02 An rms_rules / limits table

In place of a single blocklist, once slide 11's limits exist.

03 Connection pooling

PgBouncer, once order flow goes past a handful of concurrent connections.

04 Push, not poll

Order / fill / position updates over the WebSocket channel that already exists (slide 7) — it just needs more event types, replacing the current 3-second REST poll.

05 Redis pub/sub for WS fan-out

Needed once the API runs on more than one instance — today's WS server only works single-process.

06 Real secrets management

Vault or a cloud secrets manager before the JWT secret and SMTP password live anywhere near production — a local .env file is correct for dev, not for prod.

07 Backup & retention policy

Point-in-time recovery on Postgres; a TTL on ClickHouse tick data, since raw ticks are normally aged out once rolled-up bars are computed.

What I need from you to start the next phase

Which venue or broker feed do we integrate first?

That choice drives the tick schema and the order-routing adapter in slide 10.

Real RMS limits, by role and company.

I can build the engine in slide 11 — the margin, exposure, and position numbers are a business call, not an engineering one.

Where do the real Postgres and ClickHouse instances live?

Same local shape, or a managed service — affects the pooling and backup plan in slide 12.

Proposed sequence.

Feed integration → fills/execution model → RMS limits → ops hardening, matching slides 10 through 12 in order.